LAPORAN UJIAN TENGAH SEMESTER

KONSTRUKSI DAN EVOLUSI PERANGKAT LUNAK

SISTEM MANAJEMEN ASET LABORATORIUM DENGAN CI/CD DAN CONTAINERIZATION



Disusun oleh:

Nama	: Hafidz Rizqullah Prasetya
NIM	: 24/535493/SV/24243
Kelas	: PL5A1
Dosen Pengampu	: Dr. Muhammad Fakhrurrifqi, S.Kom., M.Cs.
	Galih Malela Damaraji, S.Pd., M.Eng.

PROGRAM STUDI D-IV

TEKNOLOGI REKAYASA PERANGKAT LUNAK

DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA

SEKOLAH VOKASI

UNIVERSITAS GADJAH MADA

YOGYAKARTA

2026

Daftar Isi

Daftar Isi **2**

Tujuan Ujian **3**

Dasar Teori **4**
2.1 Arsitektur RESTful API dan Otentikasi Token 4
2.2 Aplikasi Halaman Tunggal dan Klien HTTP Reaktif 4
2.3 Manajemen Repositori Git dan Proteksi Cabang 5
2.4 Otomatisasi Alur Kerja dengan GitHub Actions 5
2.5 Kontainerisasi dan Optimasi Dockerfile Multi-Stage 5

Hasil dan Pembahasan **6**
3.1 Backend Laravel RESTful API (CPMK 1) 6
3.1.1 Perancangan Skema Data dan Model 6
3.1.2 Otentikasi Token Laravel Sanctum 6
3.1.3 Implementasi Endpoint dan Status Code HTTP 7
3.1.4 Koleksi Postman dan Pengujian Otomatis 7
3.2 Frontend Vue.js dan Axios (CPMK 1) 9
3.2.1 Konfigurasi Variabel Lingkungan 9
3.2.2 Antarmuka Masuk Pengguna dan Manajemen Token 10
3.2.3 Dasbor Metrik dan Operasi CRUD Aset 10
3.3 Manajemen Repositori GitHub dan CI Pipeline (CPMK 2) 12
3.3.1 Struktur Repositori dan Strategi Percabangan 12
3.3.2 Konfigurasi Branch Protection Rules 13
3.3.3 Pipeline Continuous Integration (CI) 13
3.4 Continuous Deployment ke GHCR (CPMK 3) 14
3.4.1 Pembatasan Eksekusi Deploy pada Cabang Utama 14
3.4.2 Manajemen Rahasia dengan GitHub Secrets 14
3.4.3 Tag Citra Berdasarkan Commit SHA dan Registri GHCR 15
3.5 Penyusunan Dockerfile Backend dan Frontend (CPMK 4 dan CPMK 5) . 17
3.5.1 Dockerfile Backend dengan Optimasi Layer Caching 17
3.5.2 Dockerfile Frontend Multi-Stage Build 18

Kesimpulan **20**

Daftar Pustaka **21**

Tujuan Ujian

Sesuai arahan pada lembar soal Ujian Tengah Semester mata kuliah *Konstruksi dan Evolusi Perangkat Lunak* Semester Gasal Tahun Akademik 2026/2027, tujuan pengerjaan ujian ini adalah:

1. Mahasiswa mampu merancang dan mengimplementasikan layanan backend RESTful API berbasis Laravel dengan tema mandiri yang dilengkapi otentikasi token Laravel Sanctum, operasi CRUD lengkap, dan kesesuaian kode status HTTP 201, 422, serta 401 (CPMK 1).

2. Mahasiswa mampu membangun antarmuka web interaktif berbasis Vue.js 3 dan Axios yang terhubung ke backend, menyimpan token otentikasi pada penyimpanan lokal, mengelola data aset, dan membaca alamat API melalui variabel lingkungan (CPMK 1).

3. Mahasiswa mampu mengelola repositori publik Git di GitHub dengan struktur `backend/` dan `frontend/`, menerapkan aturan proteksi cabang `main` dan `dev`, serta menyusun alur pengujian otomatis menggunakan GitHub Actions (CPMK 2).

4. Mahasiswa mampu menerapkan alur penerapan berkelanjutan (*Continuous Deployment*) pada GitHub Actions yang dibatasi hanya untuk cabang `main`, mengamankan konfigurasi sensitif dengan GitHub Secrets, serta mempublikasikan citra kontainer ke GitHub Container Registry (`ghcr.io`) menggunakan tag commit SHA (CPMK 3).

5. Mahasiswa mampu menyusun berkas *Dockerfile* backend dengan optimasi *layer caching* dependensi Composer, serta berkas *Dockerfile* frontend dengan teknik *multi-stage build* berbasis Nginx yang bebas dari direktori dependensi kerja (CPMK 4 dan CPMK 5).

Dasar Teori

2.1 Arsitektur RESTful API dan Otentikasi Token

Arsitektur REST (*Representational State Transfer*) mengandalkan protokol HTTP standar untuk pertukaran representasi sumber daya secara terstruktur. Layanan REST bersifat *stateless*, yang berarti setiap permintaan dari klien harus memuat seluruh konteks otentikasi yang diperlukan tanpa bergantung pada sesi yang disimpan di memori peladen [1].

Laravel Sanctum menyediakan mekanisme penerbitan token akses pribadi (*Personal Access Token*) yang efisien untuk aplikasi halaman tunggal (SPA) maupun klien mandiri. Ketika pengguna berhasil melakukan verifikasi kredensial, peladen menghasilkan string acak berbobot kriptografis dan menyimpannya dalam bentuk hash di basis data. Klien menyimpan nilai teks biasa dari token tersebut dan menyertakannya pada tajuk HTTP `Authorization: Bearer <token>` pada setiap permintaan berikutnya. Penerapan kode status HTTP baku menjadi penanda kepatuhan arsitektur, seperti 201 saat data berhasil dibuat, 422 saat validasi masukan tidak terpenuhi, dan 401 saat permintaan tidak menyertakan identitas otentikasi yang sah [1].

2.2 Aplikasi Halaman Tunggal dan Klien HTTP Reaktif

Kerangka kerja Vue.js 3 mengusung paradigma reaktif berbasis komponen yang memungkinkan pembaruan antarmuka pengguna secara modular tanpa perlu memuat ulang seluruh halaman peramban [2].

Integrasi antara komponen visual dan peladen REST dijembatani oleh pustaka Axios. Melalui mekanisme pencegat permintaan (*request interceptor*), token yang tersimpan di `localStorage` dapat disisipkan secara konsisten pada tajuk setiap panggilan jaringan. Sebaliknya, pencegat respons (*response interceptor*) bertugas mendeteksi kode status 401 guna membersihkan data sesi lokal ketika otentikasi kedaluwarsa. Standar rekayasa perangkat lunak mengharuskan alamat URL peladen dipisahkan dari logika komponen dan dibaca melalui berkas konfigurasi lingkungan (*environment variable*) [3].

2.3 Manajemen Repositori Git dan Proteksi Cabang

Dalam metodologi pengembangan perangkat lunak modern, cabang `main` berfungsi sebagai representasi kode produksi yang stabil, sedangkan cabang `dev` digunakan untuk konsolidasi fitur yang sedang dikembangkan. Pekerjaan penambahan modul baru diisolasi pada cabang fitur dan diintegrasikan melalui *Pull Request* (PR) [4].

Fitur *Branch Protection Rules* di GitHub memungkinkan penegakan kebijakan integrasi dengan melarang pengunggahan langsung (*direct push*). Kebijakan ini mewajibkan seluruh pengujian terotomatisasi (*status checks*) berstatus lulus sebelum perubahan diizinkan untuk digabungkan, sehingga mencegah masuknya kesalahan regresi ke cabang utama [4].

2.4 Otomatisasi Alur Kerja dengan GitHub Actions

GitHub Actions menyediakan infrastruktur otomatisasi siklus hidup perangkat lunak yang merespons berbagai kejadian pada repositori. Alur kerja didefinisikan dalam format deklaratif YAML pada direktori `.github/workflows/` [5].

Pemisahan tugas antara pengujian backend (*backend test*) dan kompilasi frontend (*frontend build*) memungkinkan eksekusi secara paralel guna mempercepat umpan balik. Pada tahap lanjutan, tahap penerapan (*deploy job*) dikonfigurasi secara bersyarat sehingga hanya dieksekusi saat kode berhasil digabungkan ke cabang `main`, sedangkan pada cabang lain alur kerja otomatis berhenti setelah pengujian selesai [5].

2.5 Kontainerisasi dan Optimasi Dockerfile Multi-Stage

Teknologi kontainer Docker mengisolasi aplikasi beserta pustaka pendukungnya ke dalam satu kesatuan citra yang dapat dijalankan secara konsisten di berbagai lingkungan [6].

Penyusunan berkas *Dockerfile* yang efisien memanfaatkan prinsip pengurutan lapisan (*layer caching*). Dengan menyalin berkas dependensi seperti `composer.json` dan `package.json` sebelum menyalin kode sumber aplikasi, tahap pengunduhan pustaka tidak perlu diulang selama manifes dependensi tidak berubah. Pada frontend, teknik *multi-stage build* memisahkan tahap kompilasi berbasis Node.js dari tahap penyajian statis berbasis peladen Nginx yang berukuran ringkas, sehingga berkas sementara dan dependensi pembangun tidak ikut terbawa ke citra produksi [6, 7].

Hasil dan Pembahasan

3.1 Backend Laravel RESTful API (CPMK 1)

Pengerjaan soal nomor 1 difokuskan pada perancangan dan implementasi layanan RESTful API menggunakan kerangka kerja Laravel dengan tema Sistem Manajemen Aset Laboratorium. Sistem ini menangani inventaris peralatan komputasi, perangkat jaringan, dan instrumen laboratorium di lingkungan Departemen Teknik Elektro dan Informatika Sekolah Vokasi UGM.

3.1.1 Perancangan Skema Data dan Model

Sumber daya utama pada backend adalah `Asset`. Tabel `assets` dirancang untuk menyimpan atribut nama perangkat, kode inventaris unik, kategori, status operasional, nilai perolehan, dan lokasi laboratorium. Struktur migrasi basis data ditunjukkan pada Listing 1.

```
Schema::create('assets', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->string('code')->unique();
    $table->string('category');
    $table->string('status')->default('tersedia');
    $table->decimal('price', 15, 2);
    $table->string('location')->nullable();
    $table->timestamps();
});
```

Listing 1: Migrasi skema tabel assets pada backend Laravel.

Model `Asset` dilengkapi atribut `$fillable` guna mencegah anomali *mass assignment*, serta *type casting* agar atribut harga terkonversi menjadi format desimal presisi pada respons JSON.

3.1.2 Otentikasi Token Laravel Sanctum

Modul otentikasi dikelola oleh `AuthController`. Pengguna masuk ke sistem melalui rute publik `POST /api/login`. Jika kredensial cocok, peladen menerbitkan token teks biasa

menggunakan metode `createToken()` dari trait `HasApiTokens`. Token tersebut dikembalikan ke klien bersama data identitas dasar pengguna.

Seluruh rute sumber daya dilindungi oleh middleware `auth:sanctum`. Apabila rute privat diakses tanpa menyertakan token atau dengan token yang tidak sah, penanganan pengecualian pada `bootstrap/app.php` secara otomatis mengembalikan respons JSON dengan status **401 Unauthorized**. Rute `POST /api/logout` mencabut token aktif dari basis data sehingga token tersebut tidak dapat digunakan lagi.

3.1.3 Implementasi Endpoint dan Status Code HTTP

Operasi data pada `AssetController` mengembalikan kode status HTTP standar yang presisi:

1. `200 OK`: Dikembalikan saat memuat daftar aset (`GET /api/assets`), detail satu aset (`GET /api/assets/{id}`), memperbarui data (`PUT /api/assets/{id}`), dan menghapus data (`DELETE /api/assets/{id}`).

2. `201 Created`: Dikembalikan saat pembuatan entitas aset baru berhasil disimpan ke basis data (`POST /api/assets`).

3. `422 Unprocessable Content`: Dikembalikan saat data yang dikirimkan tidak memenuhi aturan validasi (seperti nama kosong, kode duplikat, atau format harga negatif).

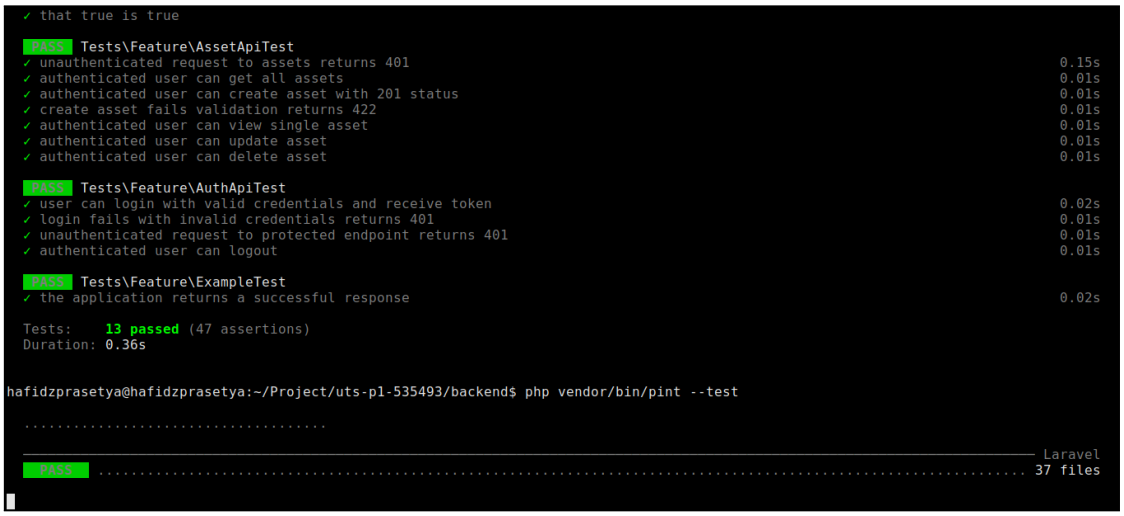
4. `401 Unauthorized`: Dikembalikan saat rute privat diakses tanpa Bearer Token yang sah.

3.1.4 Koleksi Postman dan Pengujian Otomatis

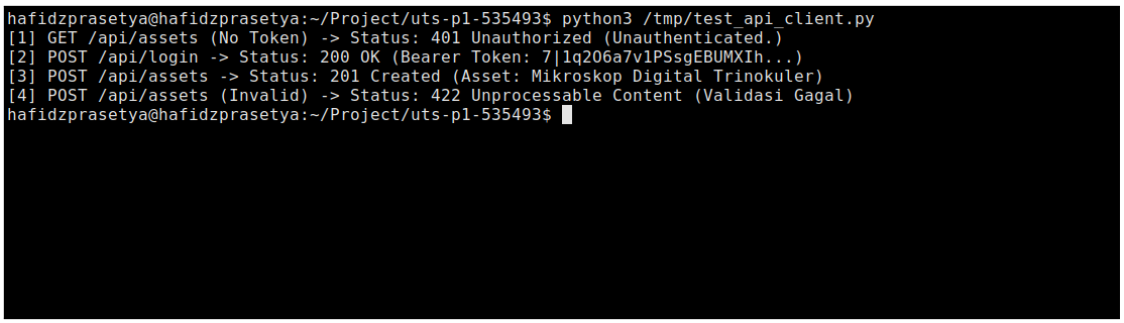
Seluruh rute API telah diuji menggunakan koleksi Postman yang diekspor ke dalam repositori pada berkas `backend/postman/uts_kepl_asset_api.postman_collection.json`. Koleksi ini mencakup skrip uji otomatis yang menyimpan token sesi ke variabel koleksi secara instan setelah panggilan login berhasil.

Keandalan backend diverifikasi secara terotomatisasi melalui berkas pengujian fitur (*Feature Tests*) pada `tests/Feature/AuthApiTest.php` dan `tests/Feature/AssetApiTest.php`. Seluruh 13 pengujian dengan 47 asersi berhasil lulus secara sempurna, disertai verifikasi kerapian kode menggunakan Laravel Pint, sebagaimana ditunjukkan pada Gambar 1.

Verifikasi kepatuhan seluruh kode status HTTP secara langsung melalui skrip pengujian terminal Linux ditampilkan pada Gambar 2.



Gambar 1: Hasil eksekusi pengujian otomatis PHPUnit dan pemeriksaan gaya kode Laravel Pint pada terminal Linux.



Gambar 2: Verifikasi pemenuhan kode status HTTP 401, 200, 201, dan 422 melalui pemanggilan klien API di terminal Linux.

3.2 Frontend Vue.js dan Axios (CPMK 1)

Pengerjaan soal nomor 2 mencakup pembangunan antarmuka web modern menggunakan Vue.js 3, bundler Vite, dan pustaka HTTP Axios.

3.2.1 Konfigurasi Variabel Lingkungan

Untuk memenuhi ketentuan soal 2.c, alamat basis API tidak ditulis mati di dalam kode komponen. Alamat tersebut dikonfigurasi melalui berkas `.env` dan diakses melalui `import.meta.env.VITE_API_BASE_URL`, seperti ditunjukkan pada Listing 2.

```
import axios from 'axios';

const apiBaseUrl = import.meta.env.VITE_API_BASE_URL || 'http://localhost:8000/api';

const apiClient = axios.create({
  baseURL: apiBaseUrl,
  headers: {
    'Accept': 'application/json',
    'Content-Type': 'application/json',
  },
});

apiClient.interceptors.request.use(
  (config) => {
    const token = localStorage.getItem('auth_token');
    if (token) {
      config.headers.Authorization = `Bearer ${token}`;
    }
    return config;
  },
  (error) => Promise.reject(error)
);

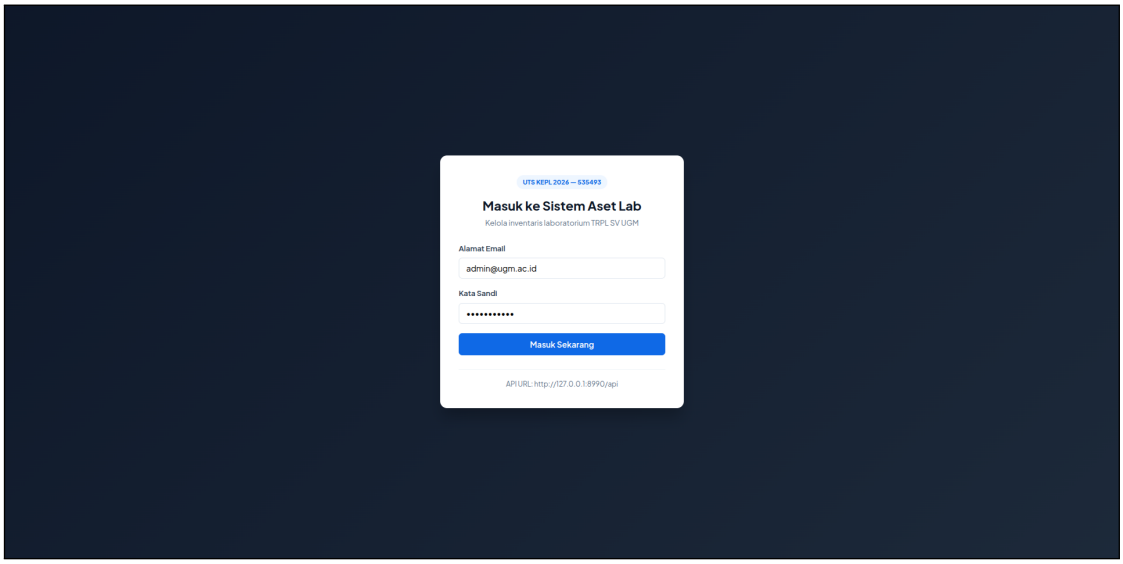
apiClient.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response && error.response.status === 401) {
      localStorage.removeItem('auth_token');
      localStorage.removeItem('auth_user');
      window.dispatchEvent(new CustomEvent('auth:unauthorized'));
    }
    return Promise.reject(error);
  }
);

export default apiClient;
export { apiBaseUrl };
```

Listing 2: Konfigurasi klien Axios dengan pembacaan variabel lingkungan.

3.2.2 Antarmuka Masuk Pengguna dan Manajemen Token

Saat pengguna membuka aplikasi, sistem memeriksa ketersediaan sesi di penyimpanan lokal. Jika sesi belum tersedia, formulir login ditampilkan sebagaimana terlihat pada Gambar 3.



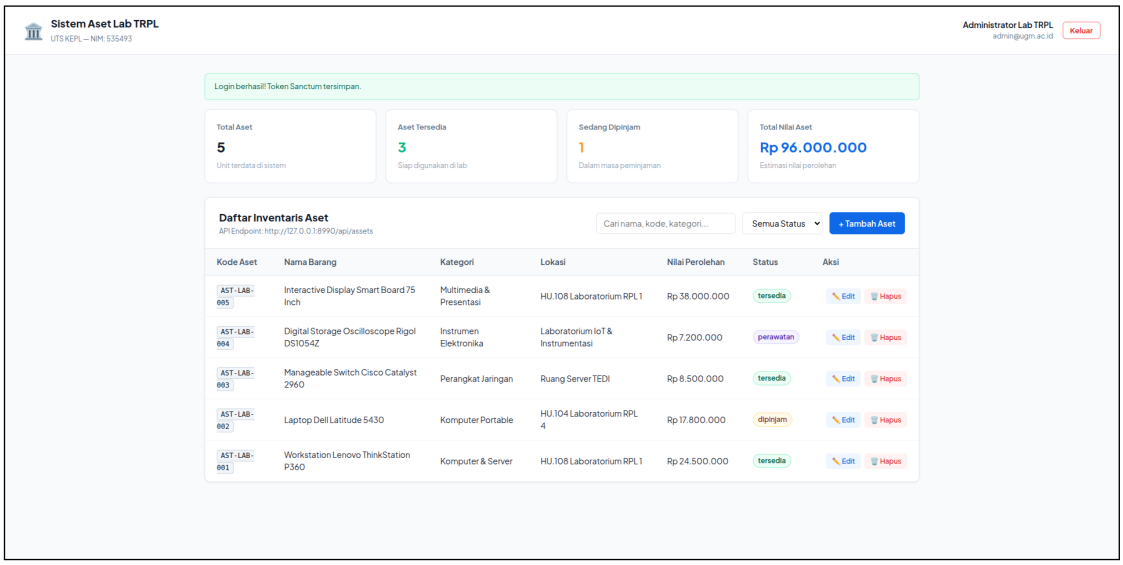
Gambar 3: Halaman login antarmuka web Vue.js dengan pembacaan dinamis URL API dari variabel lingkungan.

Ketika formulir dikirimkan, token yang diterima dari backend disimpan ke dalam `localStorage` dengan kunci `auth_token`. Pencegat Axios secara otomatis menyertakan token ini pada tajuk otentikasi di setiap permintaan berikutnya.

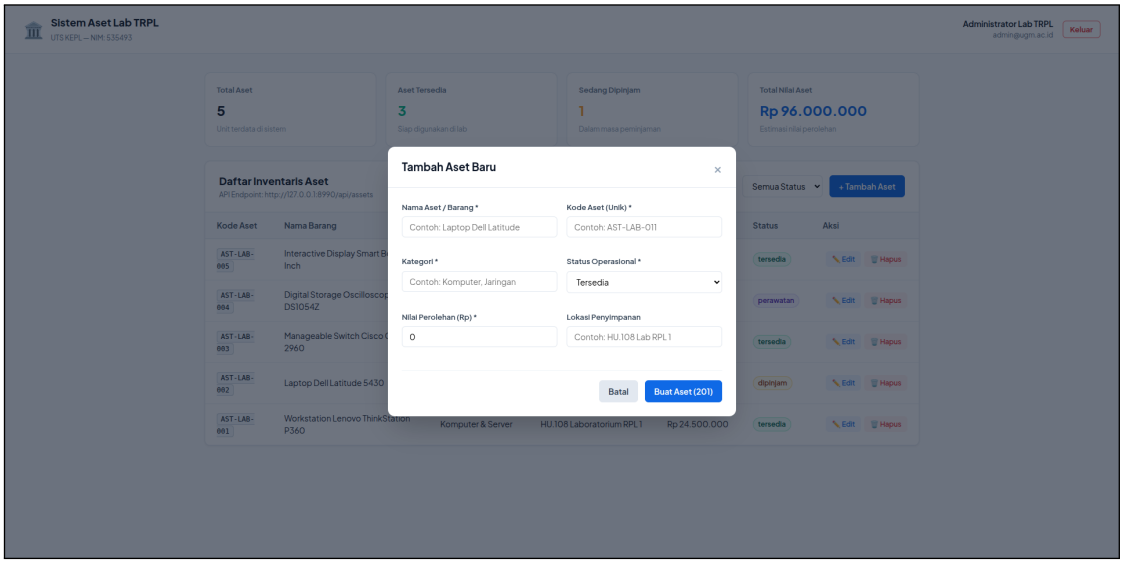
3.2.3 Dasbor Metrik dan Operasi CRUD Aset

Setelah otentikasi berhasil, pengguna diarahkan ke dasbor utama. Dasbor menampilkan kartu metrik ringkasan inventaris, kolom pencarian teks, filter status, serta tabel daftar aset lengkap. Tampilan dasbor disajikan pada Gambar 4.

Penambahan aset baru dilakukan melalui jendela modal interaktif. Modal ini mengirimkan data ke endpoint `POST /api/assets`. Ketika peladen merespons dengan status 201 Created, daftar aset pada tabel langsung diperbarui secara reaktif tanpa perlu memuat ulang peramban. Tampilan modal penambahan aset ditunjukkan pada Gambar 5.



Gambar 4: Dasbor manajemen inventaris aset laboratorium pada antarmuka web Vue.js.



Gambar 5: Modal interaktif penambahan data aset laboratorium baru pada antarmuka frontend.

3.3 Manajemen Repositori GitHub dan CI Pipeline (CPMK 2)

Sesuai soal nomor 3, seluruh kode sumber dikelola pada repositori publik bernama `uts-p1-535493` di akun GitHub pribadi (`https://github.com/hafidzrizqullahprasetya/uts-p1-535493`). Repositori memuat folder `backend/` dan `frontend/`, serta menerapkan aturan percabangan yang terisolasi.

3.3.1 Struktur Repositori dan Strategi Percabangan

Repositori diatur dengan tiga tingkatan cabang:

1. `main`: Cabang produksi utama yang dilindungi secara ketat.
2. `dev`: Cabang integrasi pengembangan pra-produksi.
3. `feat/backend-and-frontend`: Cabang fitur untuk pengembangan modul backend dan frontend.

Seluruh perubahan kode dari cabang fitur diintegrasikan ke cabang `dev` melalui Pull Request nomor 1. Setelah diverifikasi lulus oleh pengujian CI, cabang `dev` digabungkan ke cabang `main` melalui Pull Request nomor 2. Riwayat komit dan pohon percabangan pada terminal Linux ditampilkan pada Gambar 6.

```
hafidzprasetya@hafidzprasetya:~/Project/uts-p1-535493$ git status
On branch main
Your branch is up to date with 'origin/main'.

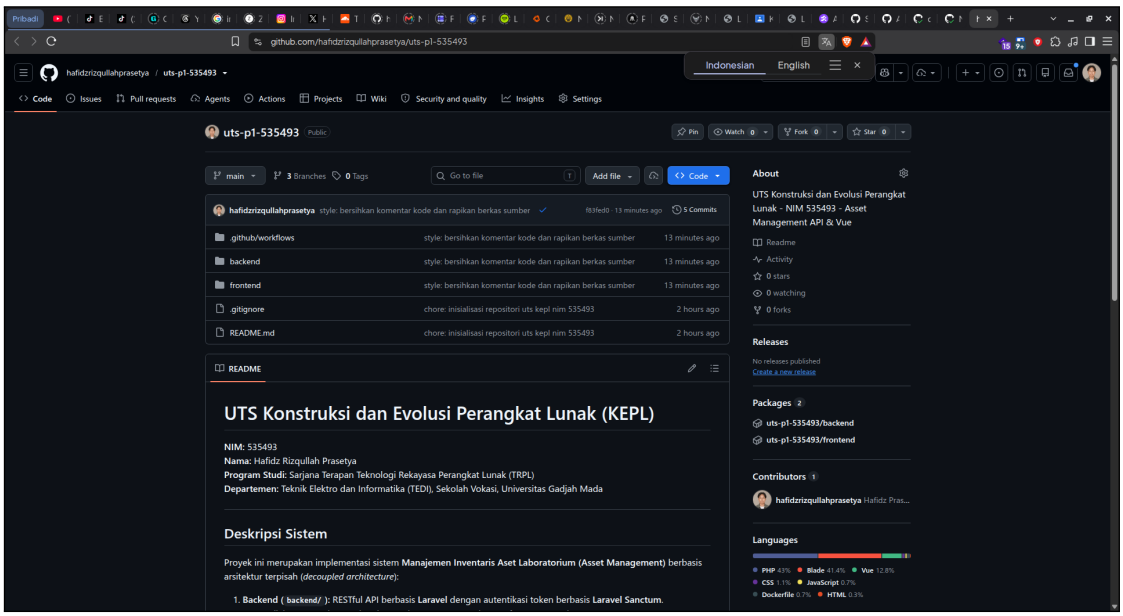
nothing to commit, working tree clean

hafidzprasetya@hafidzprasetya:~/Project/uts-p1-535493$ git branch -a
  dev
  feat/backend-and-frontend
* main
  remotes/origin/dev
  remotes/origin/feat/backend-and-frontend
  remotes/origin/main

hafidzprasetya@hafidzprasetya:~/Project/uts-p1-535493$ git log --graph --oneline -n 6
* f83fed0 (HEAD -> main, origin/main) style: bersihkan komentar kode dan rapikan berkas sumber
*   085dd03 Merge pull request #2 from hafidzrizqullahprasetya/dev
|\
| * 7d1ddd6 (origin/dev, dev) Merge pull request #1 from hafidzrizqullahprasetya/feat/backend-and-frontend
|/|
| * 50ce732 (origin/feat/backend-and-frontend, feat/backend-and-frontend) feat: implementasi backend laravel sanctum dan frontend vue
|/
* 07c7b52 chore: inisialisasi repositori uts kepl nim 535493
```

Gambar 6: Pemeriksaan status cabang Git dan riwayat graf penggabungan Pull Request pada terminal Linux.

Tampilan beranda repositori publik di GitHub disajikan pada Gambar 7, yang memperlihatkan struktur direktori `backend/` dan `frontend/` yang terkelola rapi.



Gambar 7: Tampilan repositori publik `uts-p1-535493` pada platform GitHub.

3.3.2 Konfigurasi Branch Protection Rules

Cabang `main` dan `dev` diproteksi untuk menolak pengunggahan langsung (*direct push*). Aturan proteksi mewajibkan:

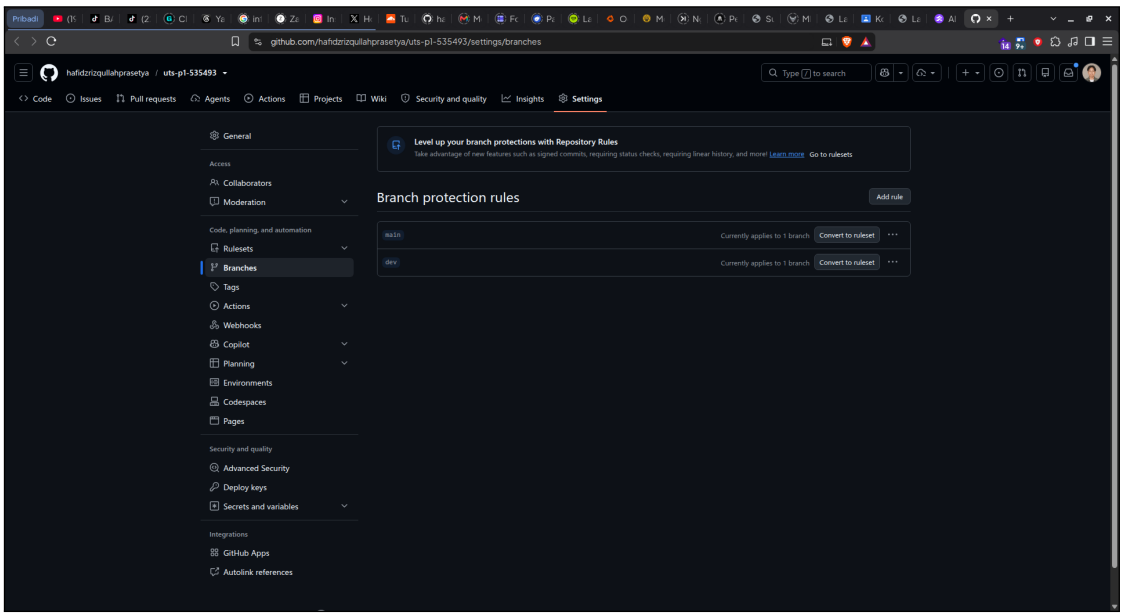
1. Integrasi hanya dapat dilakukan melalui mekanisme Pull Request.
2. Pemeriksaan status (*status checks*) harus lulus sebelum penggabungan, yaitu pekerjaan `Backend Test` dan `Frontend Build`.
3. Opsi sinkronisasi cabang mutakhir (*strict check*) aktif.

Bukti pengaturan Branch Protection Rules pada antarmuka GitHub ditunjukkan pada Gambar 8.

3.3.3 Pipeline Continuous Integration (CI)

Alur kerja integrasi berkelanjutan didefinisikan pada berkas `.github/workflows/ci.yml`. Berkas ini memuat dua pekerjaan pengujian yang berjalan secara paralel:

1. **Backend Test:** Menyiapkan lingkungan PHP 8.4, memasang dependensi Composer dari cache, menginisialisasi basis data SQLite lokal, menjalankan migrasi, dan mengeksekusi `php artisan test`.
2. **Frontend Build:** Menyiapkan lingkungan Node.js 20, memasang pustaka menggunakan `npm ci`, dan memverifikasi kompilasi aset melalui `npm run build`.



Gambar 8: Pengaturan Branch Protection Rules untuk cabang `main` dan `dev` pada repositori GitHub.

3.4 Continuous Deployment ke GHCR (CPMK 3)

Soal nomor 4 mewajibkan penambahan pekerjaan penerapan (*deploy job*) pada alur kerja GitHub Actions guna mempublikasikan citra kontainer ke GitHub Container Registry (`ghcr.io`).

3.4.1 Pembatasan Eksekusi Deploy pada Cabang Utama

Pekerjaan penerapan dikonfigurasi agar hanya berjalan saat terjadi peristiwa pengunggahan (*push*) ke cabang `main`. Pada pengujian Pull Request atau cabang `dev`, pekerjaan deploy dilewati secara otomatis. Potongan logika eksekusi tersebut ditunjukkan pada Listing 3.

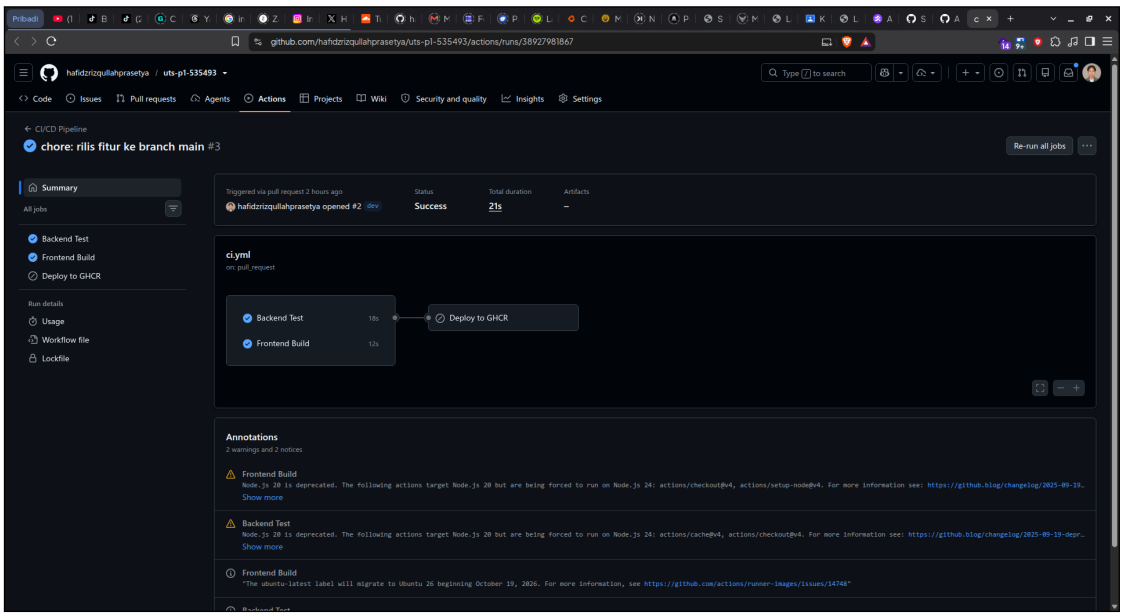
```
deploy:
  name: Deploy to GHCR
  needs: [backend-test, frontend-build]
  if: github.event_name == 'push' && github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest
```

Listing 3: Konfigurasi bersyarat pada pekerjaan deploy di ci.yml.

Bukti eksekusi alur kerja pada Pull Request cabang `dev` ditunjukkan pada Gambar 9, di mana pekerjaan pengujian backend dan kompilasi frontend berhasil diselesaikan, sedangkan pekerjaan deploy dilewati (*skipped*).

3.4.2 Manajemen Rahasia dengan GitHub Secrets

Nilai konfigurasi sensitif seperti `APP_KEY` dan `DB_PASSWORD` disimpan pada menu *Settings* → *Secrets and variables* → *Actions* di repositori GitHub. Variabel ini disuntikkan ke



Gambar 9: Riwayat eksekusi GitHub Actions pada Pull Request cabang dev yang melewati tahap deploy.

dalam instruksi pembangunan kontainer sebagai argumen pembangun (*build-args*) tanpa pernah tersimpan secara terbuka pada repositori publik.

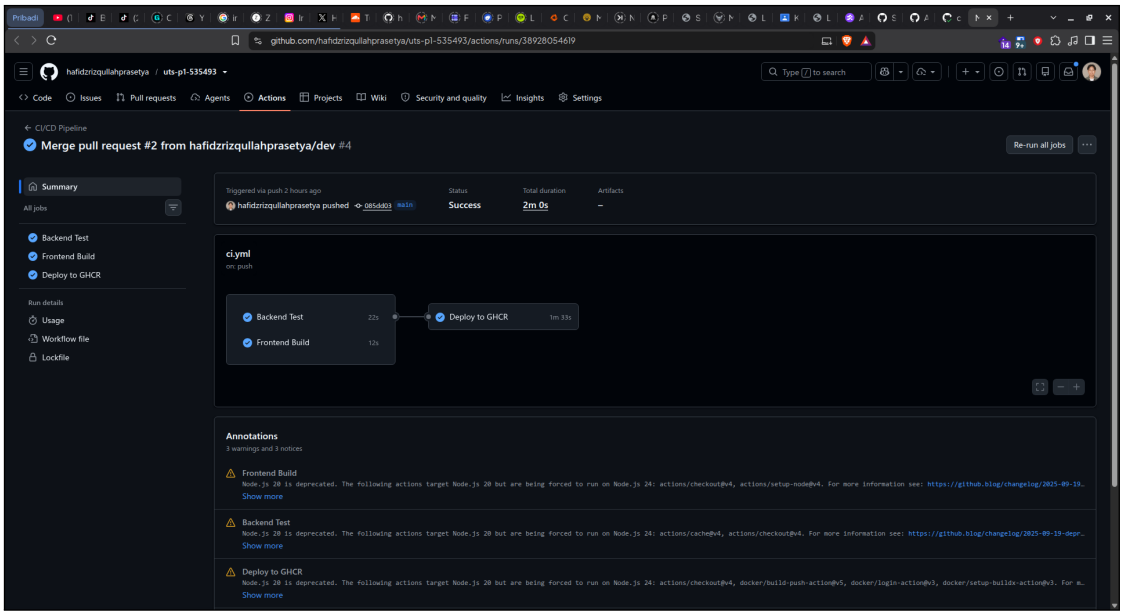
3.4.3 Tag Citra Berdasarkan Commit SHA dan Registri GHCR

Pekerjaan penerapan melakukan otentikasi ke `ghcr.io` menggunakan token bawaan `GITHUB_TOKEN` dengan izin `packages: write`. Citra kontainer backend dan frontend dibangun dan diunggah dengan tag yang merujuk pada hash commit penuh (`${{ github.sha }}`), bukan tag `latest`. Pemberian tag unik tersebut menjamin ketertelusuran artefak rilis dengan versi kode sumber yang menyusunnya.

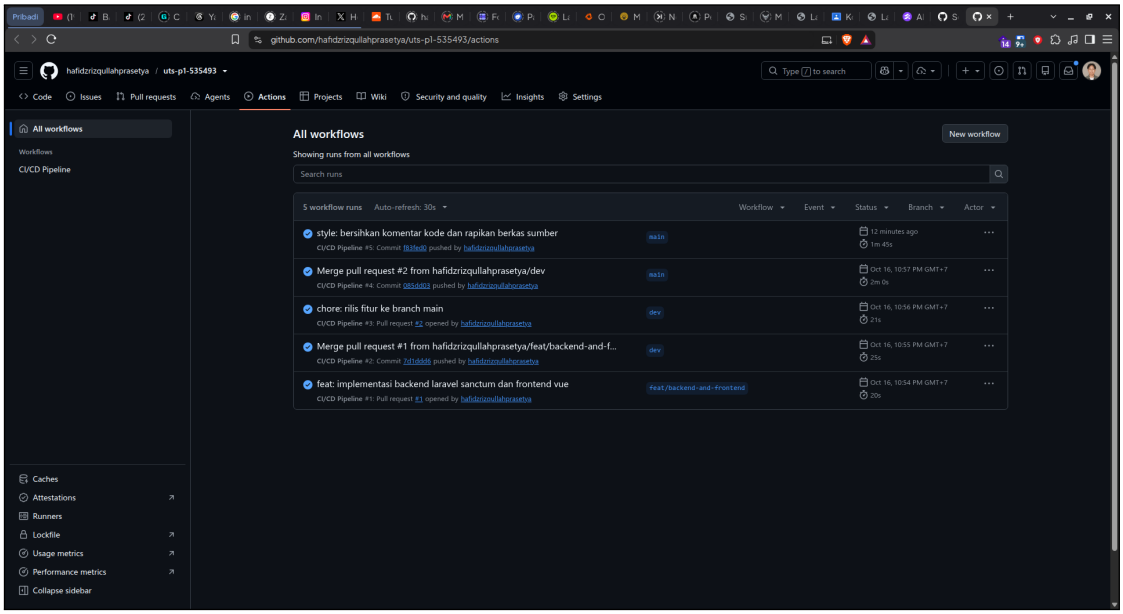
Gambar 10 memperlihatkan eksekusi alur kerja pada cabang `main` yang berhasil menyelesaikan seluruh pekerjaan secara berurutan dengan status hijau sempurna.

Seluruh riwayat pengujian dan penerapan terangkum pada tab Actions repositori, sebagaimana disajikan pada Gambar 11.

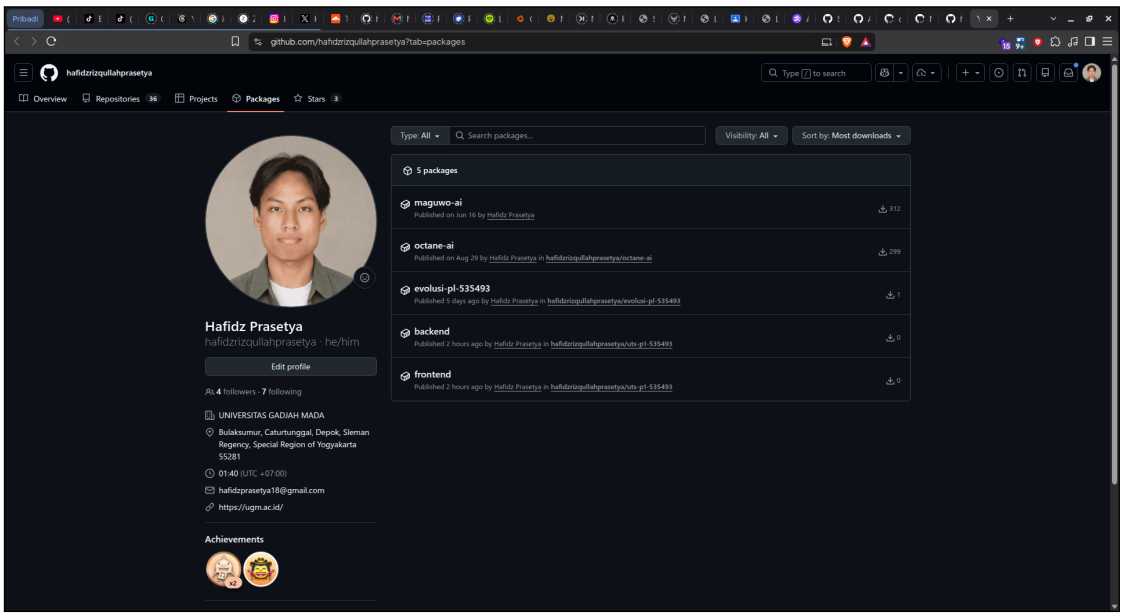
Hasil publikasi kedua citra kontainer pada GitHub Container Registry ditampilkan pada Gambar 12. Kedua paket (`uts-p1-535493/backend` dan `uts-p1-535493/frontend`) terdaftar di bawah namespace repositori dan siap diunduh untuk kebutuhan operasional peladen.



Gambar 10: Eksekusi alur kerja CI/CD pada cabang `main` dengan tahap deploy yang sukses ke GHCR.



Gambar 11: Riwayat lengkap alur kerja pada tab Actions repositori GitHub.



Gambar 12: Halaman registri paket GitHub Container Registry memuat citra backend dan frontend ber-tag commit SHA.

3.5 Penyusunan Dockerfile Backend dan Frontend (CPMK 4 dan CPMK 5)

Soal nomor 5 menuntut penyusunan berkas *Dockerfile* untuk backend dan frontend dengan standar optimasi kontainerisasi industri.

3.5.1 Dockerfile Backend dengan Optimasi Layer Caching

Berkas `backend/Dockerfile` menggunakan citra dasar resmi `php:8.4-cli-alpine` yang eksplisit dan bebas dari tag `latest`. Untuk memanfaatkan mekanisme *cache* lapisan Docker secara optimal, berkas manifes dependensi `composer.json` dan `composer.lock` disalin terlebih dahulu, diikuti proses instalasi pustaka Composer. Kode sumber aplikasi baru disalin pada lapisan berikutnya, seperti disajikan pada Listing 4.

```
FROM php:8.4-cli-alpine

WORKDIR /var/www/html

RUN apk add --no-cache \
    curl \
    git \
    unzip \
    libzip-dev \
    sqlite-dev \
    libpng-dev \
    oniguruma-dev \
    libxml2-dev \
```

```
    linux-headers \
    && docker-php-ext-install -j$(nproc) \
    pdo \
    pdo_sqlite \
    mbstring \
    xml \
    zip \
    bcmath

COPY --from=composer:2.8 /usr/bin/composer /usr/bin/composer

COPY composer.json composer.lock ./
RUN composer install --no-dev --no-scripts --no-autoloader --prefer-dist

COPY . .

RUN composer dump-autoload --optimize \
    && cp -n .env.example .env || true \
    && touch database/database.sqlite \
    && php artisan key:generate --force \
    && chown -R www-data:www-data storage bootstrap/cache database

EXPOSE 8000

CMD ["php", "artisan", "serve", "--host=0.0.0.0", "--port=8000"]
```

Listing 4: Berkas backend/Dockerfile dengan optimasi layer caching Composer.

3.5.2 Dockerfile Frontend Multi-Stage Build

Berkas `frontend/Dockerfile` mengimplementasikan arsitektur *multi-stage build*. Pada tahap pertama (`builder`), citra menggunakan `node:20-alpine` untuk memasang pustaka dependensi dan mengompilasi aset web produksi melalui `npm run build`. Pada tahap kedua, citra produksi menggunakan peladen `nginx:1.27-alpine`. Berkas hasil kompilasi dari direktori `/app/dist` disalin ke direktori publik Nginx (`/usr/share/nginx/html`), sedangkan direktori `node_modules` ditinggalkan pada tahap pembangun. Pendekatan ini menghasilkan citra kontainer akhir yang bersih dan berukuran minimal, sebagaimana dicantumkan pada Listing 5.

```
FROM node:20-alpine AS builder

WORKDIR /app

COPY package*.json ./
RUN npm ci

COPY . .
```

```
RUN npm run build

FROM nginx:1.27-alpine

RUN rm -rf /usr/share/nginx/html/*
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=builder /app/dist /usr/share/nginx/html

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

Listing 5: Berkas frontend/Dockerfile menggunakan teknik multi-stage build.

Kesimpulan

Berdasarkan seluruh tahapan perancangan, implementasi, dan pengujian yang telah diselesaikan pada Ujian Tengah Semester mata kuliah Konstruksi dan Evolusi Perangkat Lunak, dapat ditarik beberapa kesimpulan:

1. Layanan backend berbasis Laravel RESTful API berhasil dibangun dengan tema Sistem Manajemen Aset Laboratorium. Otentikasi berbasis token Laravel Sanctum berhasil memproteksi seluruh rute privat, dengan kepatuhan kode status HTTP standar (201 Created, 422 Unprocessable Content, 401 Unauthorized, dan 200 OK) yang terbukti melalui 13 pengujian otomatis berstatus lulus dan koleksi Postman yang lengkap (CPMK 1).

2. Antarmuka frontend berbasis Vue.js 3 dan Axios berhasil mengintegrasikan alur login, persistensi token pada penyimpanan lokal, dan operasi CRUD aset secara reaktif. URL basis API berhasil dikonfigurasi secara dinamis melalui variabel lingkungan `VITE_API_BASE_URL` (CPMK 1).

3. Repositori sistem berhasil dikelola pada repositori publik GitHub `uts-p1-535493` dengan struktur direktori `backend/` dan `frontend/`. Kebijakan *Branch Protection Rules* berhasil ditegakkan pada cabang `main` dan `dev` untuk mewajibkan alur Pull Request dan verifikasi kelulusan pengujian CI sebelum kode digabungkan (CPMK 2).

4. Alur penerapan berkelanjutan (*Continuous Deployment*) berhasil dikonfigurasi pada alur kerja GitHub Actions. Pekerjaan deploy berhasil dibatasi hanya untuk kejadian push ke cabang `main`, mengamankan variabel sensitif menggunakan GitHub Secrets, dan mempublikasikan citra kontainer ke GitHub Container Registry (`ghcr.io`) dengan tag commit SHA penuh (CPMK 3).

5. Kontainerisasi sistem berhasil dioptimalkan melalui penyusunan *Dockerfile backend* dengan strategi *layer caching* dependensi Composer, serta *Dockerfile frontend* yang menerapkan arsitektur *multi-stage build* berbasis Nginx tanpa menyertakan direktori `node_modules` pada citra akhir (CPMK 4 dan CPMK 5).

Daftar Pustaka

[1] Laravel LLC, "Laravel Sanctum - Light-weight authentication for SPAs and token APIs," 2026. [Daring]. Tersedia: https://laravel.com/framework/docs/sanctum. [Diakses: 16-Okt-2026].

[2] Vue.js Core Team, "Introduction to Vue.js - The Progressive JavaScript Framework," 2026. [Daring]. Tersedia: https://vuejs.org/guide/introduction.html. [Diakses: 16-Okt-2026].

[3] Axios Contributors, "Axios HTTP Client - Getting Started," 2026. [Daring]. Tersedia: https://axios.rest/pages/getting-started/first-steps. [Diakses: 16-Okt-2026].

[4] GitHub Inc., "About Protected Branches and Pull Request Gates," 2026. [Daring]. Tersedia: https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches. [Diakses: 16-Okt-2026].

[5] GitHub Inc., "Understanding GitHub Actions - Workflow Automation and CI/CD," 2026. [Daring]. Tersedia: https://docs.github.com/en/actions/get-started/understand-github-actions. [Diakses: 16-Okt-2026].

[6] Docker Inc., "Multi-stage Builds - Optimize Container Image Sizes and Security," 2026. [Daring]. Tersedia: https://docs.docker.com/build/building/multi-stage/. [Diakses: 16-Okt-2026].

[7] Nginx Inc., "Nginx Beginner's Guide - Web Server Configuration for Static Applications," 2026. [Daring]. Tersedia: https://nginx.org/en/docs/beginners_guide.html. [Diakses: 16-Okt-2026].